Wear-Aware Routing Algorithm for Network-on-Chip Interconnects Using Linear Approximate Reinforcement Learning

by

Farhan Ishraq

A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR'S OF APPLIED SCIENCE

in

The Faculty of Applied Science

(Engineering Physics)

THE UNIVERSITY OF BRITISH COLUMBIA

(Vancouver)

April 2026

© Farhan Ishraq 2026

The following individuals certify that they have read, and recommend to the Faculty of Applied Science for acceptance, the thesis entitled:

Wear-Aware Routing Algorithm for Network-on-Chip Interconnects Using Linear Approximate Reinforcement Learning

submitted by **Farhan Ishraq** in partial fulfillment of the requirements for the degree of **Bachelor of Applied Science** in **Engineering Physics**.

Examining Committee:

Andre Ivanov, Professor, Electrical and Computer Engineering, UBC
Supervisor

Abstract

As chip multiprocessors scale, Network-on-Chip (NoC) reliability becomes a critical concern due to aging mechanisms such as electromigration (EM) and hot-carrier injection (HCI), which are strongly dependent on traffic-induced switching activity and temperature. Conventional routing algorithms such as dimension-order routing (DOR) are oblivious to these effects, leading to uneven wear and reduced system lifetime. This work presents a reinforcement learning (RL)-based adaptive routing framework that incorporates aging awareness directly into routing decisions. Each router operates as a distributed RL agent using linear function approximation, with EM and HCI-based mean time to failure (MTTF) estimates integrated into the state and reward formulation. Deadlock freedom is ensured through an escape virtual channel using deterministic routing. The approach is evaluated using the gem5 Garnet simulator on an 8×8 mesh across multiple synthetic traffic patterns. Results show significant lifetime improvements, achieving up to $5.3\times$ acceleration in EM lifetime and consistent gains for HCI, while maintaining or improving packet latency by distributing traffic more evenly and delaying network saturation. This work demonstrates that RL-based routing can effectively improve NoC reliability without compromising performance.

Lay Summary

Modern computer chips rely on internal networks to move data between many processing units, but these connections can wear out over time, especially when some paths are used more than others. Traditional routing methods send data along fixed paths, which can overload certain parts of the network and shorten the chip's lifespan. This thesis explores a learning-based approach where the network adapts its routing decisions based on current conditions, spreading traffic more evenly to reduce stress on heavily used components. By combining this approach with models of how hardware wears out, the system learns to avoid paths that are likely to fail sooner. Our results show that this method can significantly extend the lifetime of the network while maintaining or even improving communication speed, highlighting a promising direction for building more reliable and efficient computer systems.

Preface

The study is based on work conducted at the Ivanov SoC Lab at UBC under the guidance of Professor Andre Ivanov. I was responsible for designing and running experiments, reviewing and presenting relevant literature, and compiling the results. Much consultation was taken with Ian Hill, Roozmehr Jalilian and Dante St. Prix, whose suggestions, insights, and expertise helped guide the project.

Table of Contents

Abstract iii

Lay Summary iv

Preface v

Table of Contents vi

List of Figures viii

List of Abbreviations ix

Acknowledgements x

1 Introduction 1
- 1.1 Problem Statement 3
- 1.2 Scope of Research 3

2 Literature Review 4
- 2.1 Introduction to Routing Algorithms in NoCs 4
- 2.2 Fault-Tolerant Routing Algorithm in NoC 5
- 2.3 Aging-Aware Routing Algorithm 6
- 2.4 Gaps in Prior Studies 7

3 Aging-Aware Routing Algorithm System Design 8
- 3.1 Dynamic Temperature Variation 8
- 3.2 Process Variation (PV) 9
- 3.3 Electromigration (EM) 10
- 3.4 Hot-Carrier Injection (HCI) 10
- 3.5 Reinforcement Learning-Based Routing Policy 12
- 3.6 Deadlock Avoidance 15

4 Evaluation 17
4.1 Simulation Setup 17
4.2 Garnet Synthetic Traffic and Evaluation Metrics 18
4.3 Results 19

Bibliography 26

Appendices

A Mean Acceleration Factor 29
A.1 Mean Acceleration Factor (AF) Trends Across Injection Rates 29

List of Figures

1.1 CMP Failure Modes 2

3.1 Reinforcement Learning Framework 12

4.1 Mean EM Wear in Links 19
4.2 Mean HCI Wear in Routers 20
4.3 Average Packet Latency (Transpose, Bit Reverse, Bit Rotation) 22
4.4 Average Packet Latency under Bit Rotation Traffic. 23
4.5 Average Packet Latency (Uniform, Shuffle) 24
4.6 Average Packet Latency (Tornado, Neighbor) 25

A.1 Uniform Random Traffic. 29
A.2 Shuffle Traffic. 29
A.3 Transpose Traffic. 30
A.4 Bit Reverse Traffic. 30
A.5 Bit Rotation Traffic. 30
A.6 Neighbor Traffic. 31
A.7 Tornado Traffic. 31

List of Abbreviations

CMP	Chip Multiprocessor
NoC	Network-on-Chip
EM	Electromigration
HCI	Hot-Carrier Injection
DOR	Dimension-Order Routing
MTTF	Mean Time To Failure
RL	Reinforcement Learning
TTpE	Traffic Threshold per Epoch
CAT	Centralized Aging Table
D-CAT	Distributed Centralized Aging Table
BTI	Bias Temperature Instability
PV	Process Variation
AF	Acceleration Factor
VC	Virtual Channel

Acknowledgements

First and foremost, I would like to extend my sincerest thanks to my supervisor, Andre Ivanov. Without his guidance and support, I would not have been able to pursue this path.

I am also grateful to the members of my research group who have given me valuable input throughout all stages of my work. Ian Hill, Roozmehr Jalilian, Dante St. Prix and Raul Vazquez Guerrero have been wonderful members of the group; their support has been indispensible.

Chapter 1

Introduction

Modern chip multiprocessors (CMPs) rely on Network-on-Chip (NoC) interconnects to support communication between cores, caches, memory controllers, and other peripheral devices. As system sizes scale to tens and even hundreds of cores, NoC become a central component in determining overall system performance, as they directly impact latency, throughput, and scalability [1].

With continued technology scaling and prolonged operational stresses induced by demanding CMP worloads, however, reliability has emerged as a major concern. Aging mechanisms such as electromigration (EM) in interconnects and hot-carrier injection (HCI) in transistors degrade components over time. EM primarily affects interconnect wires under high current density and temperature, while HCI degrades transistors based on switching stress [2]. In both cases, aging is driven by switching activity, meaning heavily utilized components wear out faster, especially under elevated temperature and process variation. In NoCs, these factors are directly influenced by traffic patterns, making routing decisions a key determinant of how wear accumulates across the network.

Wearout in interconnects is particularly concerning because, unlike cores, which can often tolerate failures through redundancy or task remapping, failures in the interconnect are far more critical. As illustrated in Figure 1.1, while a core failure (Fig. 1.1a) may degrade performance, router (Fig. 1.1b) or link failures (Fig. 1.1c) can disrupt connectivity, partition the network, and render large portions of the system unusable.

Conventional routing algorithms, such as dimension-order routing (DOR), are oblivious to these effects. Their deterministic nature leads to uneven traffic distribution, with central routers and links experiencing consistently higher utilization. This imbalance translates directly into accelerated aging of specific components, effectively reducing the lifetime of the entire network. Prior work on NoC reliability has largely focused on fault-tolerant

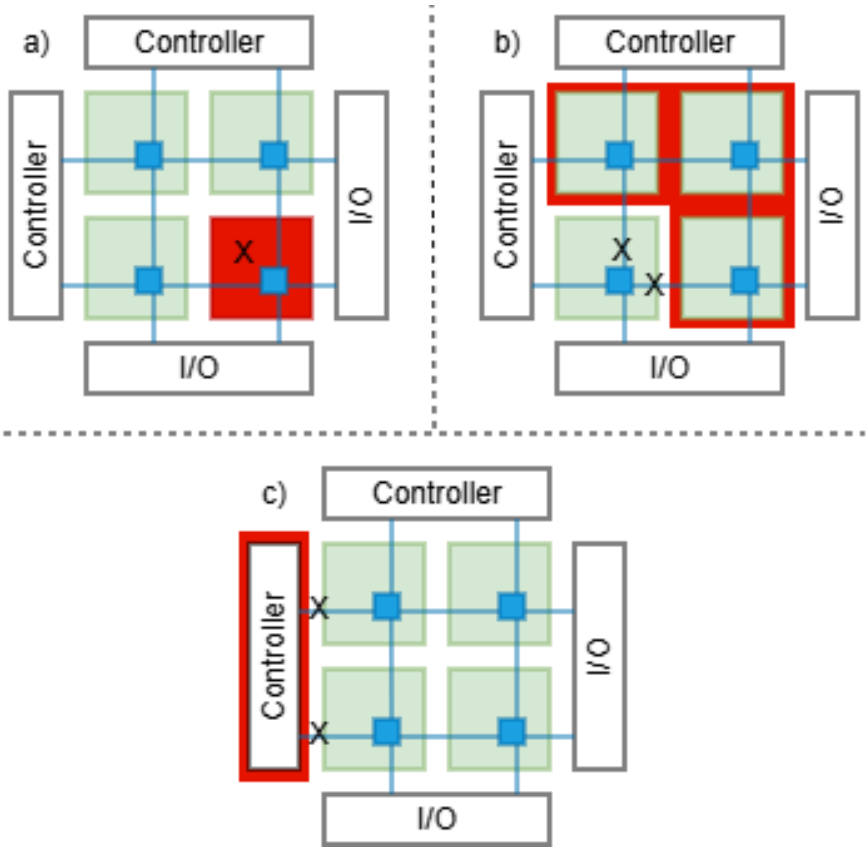


Figure 1.1: 4-core CMP with a 2D mesh NoC interconnect. (a) Core failure due to wearout. (b) Router failure due to wearout. (c) Link failure due to wearout.

routing, where the objective is to maintain connectivity in the presence of permanent faults. These approaches are inherently reactive, requiring fault detection and reconfiguration after failures occur [3] [4] [5]. In contrast, aging-aware routing aims to delay the onset of such failures by controlling how traffic is distributed over time. This class of routing algorithms is fundamentally adaptive: routing decisions are influenced by dynamic metrics such as utilization, temperature, and accumulated wear, with the goal of balancing stress across the network and extending system lifetime.

Existing aging-aware routing techniques typically rely on explicit modeling of path reliability, combining metrics such as EM- and HCI-induced mean time to failure (MTTF) across candidate routes. While effective, these ap-

proaches often require global visibility of network state and repeated path evaluation, resulting in significant computational and storage overhead. Furthermore, their reliance on fixed cost functions and heuristics limits their ability to adapt to changing traffic patterns and evolving wear conditions over long execution periods [2] [6].

1.1 Problem Statement

Current NoC routing algorithms do not account for long-term aging effects, leading to uneven wear and premature failure of critical components. Existing aging-aware approaches attempt to address this issue but often depend on "piggybacking" global information and fixed heuristics, making them difficult to scale and adapt to dynamic conditions.

This work focuses on developing a routing strategy that can adapt to changing network conditions while remaining lightweight and scalable, with the goal of improving overall system lifetime without significantly impacting performance.

1.2 Scope of Research

This thesis proposes a wear-aware routing approach based on linear approximate reinforcement learning. Instead of explicitly computing path reliability metrics, the routing policy is learned through interaction with the network, allowing the system to implicitly capture the relationship between traffic patterns and aging effects.

The key contributions of this work are:

- A reinforcement learning-based aging-aware routing algorithm
- Integration of a linear function approximation model that enables scalability and low-overhead implementation at each router
- Integration of a deadlock-free adaptive routing design using escape virtual channels
- Implementation in the gem5 Garnet NoC simulator and evaluation demonstrating improved lifetime metrics

Chapter 2

Literature Review

2.1 Introduction to Routing Algorithms in NoCs

Routing determines how packets travel through a Network-on-Chip from their source to their destination. In a typical 2D mesh NoC, each router forwards packets according to the routing algorithm implemented in its routing unit, with the overall goal of minimizing latency, sustaining throughput, and avoiding congestion and deadlock. NoC routing algorithms are broadly categorized as static or adaptive. Static routing uses fixed, predetermined paths independent of runtime conditions. Deterministic XY routing is the standard example, where packets are routed first in the X direction and then in the Y direction. Although this approach is simple and low cost, it does not react to traffic imbalance or failures, and can therefore create bottlenecks and underutilize alternative paths [6].

Adaptive routing addresses this limitation by selecting routes based on the current network state, such as congestion levels, fault status, or buffer availability. These routing decisions may rely on local information, global information, or a hybrid of both, depending on the algorithm design. For this work, we are interested in adaptive routing only, which is best understood through the taxonomy proposed in the recent survey of 2D NoC routing [7], which divides adaptive approaches into reactive, proactive, and application-specific classes. Reactive routing responds to conditions that already exist in the network, while proactive routing attempts to prevent future degradation by anticipating harmful operating conditions.

Within reactive adaptation, one important class is fault-tolerant routing, whose objective is to preserve packet delivery by rerouting traffic around failed routers or links. Within proactive adaptation, one important class is aging-aware routing, which aims to reduce long-term wear by steering traffic away from components experiencing higher stress. In other words, fault-tolerant routing responds after faults occur, whereas aging-aware routing

attempts to delay those faults by controlling the accumulation of wear over time [7]. This work focuses on extending NoC lifetime as opposed to merely recovering from failure. The remainder of this literature review concentrates on adaptive routing methods in these two categories, with particular emphasis on aging-aware routing.

2.2 Fault-Tolerant Routing Algorithm in NoC

Fault-tolerant routing is concerned with maintaining communication in the presence of faulty routers or links. Its primary objective is to minimize packet loss and maximize successful packet delivery by rerouting traffic around failed components, while keeping latency and throughput within acceptable limits.

A simple approach is the dynamic XY-YX fault-tolerant routing algorithm [8], which switches from XY routing to YX routing when faults are encountered. However, the method is fundamentally constrained by a small set of predefined routing responses. Once faults create more complex blockage patterns, the algorithm cannot reason beyond the fixed XY/YX alternatives. Similar approaches are represented by logic- and rule-based fault-tolerant routing schemes summarized in the survey [7], such as FTMR, RAFT, Hermes, and related fault-ring or contour-based methods. These approaches identify faulty regions, classify unusable nodes, or maintain local fault information to bypass failed components. The drawback of these methods is that they depend on explicit fault modeling, packet-header updates, or region management logic, all of which add control overhead and can still lead to suboptimal paths when network conditions become more complex.

Recent work has therefore explored reinforcement learning (RL) as a way to make fault-tolerant routing more adaptive. In [9], routing is modeled as a Markov Decision Process and solved using Q-learning, with routers treated as states and routing decisions as actions. The proposed method is shown to find optimal paths in the presence of both router and link faults. On average, the paper reports about 19% more delivered packets under link faults and 11% more delivered packets under router faults. However, it relies on a large Q-table that grows with the number of routers and possible fault conditions, which can lead to high memory overhead. This limitation is addressed more directly in [4]. This work extends the earlier RL-based mesh routing approach using decentralized multi-agent reinforcement learning and a two-level Q-table. The goal is to reduce the storage cost and stale-value

issues associated with large flat Q-tables, while still learning effective fault-tolerant policies. More recent work [10] shows that RL can improve network performance by learning better flit rerouting behavior.

2.3 Aging-Aware Routing Algorithm

Aging-aware routing aims to extend NoC lifetime by steering traffic away from components that are aging faster due to stress, temperature, and process variation. Unlike fault-tolerant routing, which reacts after failures occur, aging-aware routing attempts to delay failure by reducing the uneven accumulation of wear across routers and links. As the survey notes [7], the main evaluation metric in this class of work is typically MTTF, since the central objective is long-term reliability rather than immediate fault recovery.

One of the most influential works in this area is Clotho [2], which frames wear-aware routing as a path-selection problem based on the lifetime of candidate routes. Clotho models both EM and HCI, while also incorporating temperature and process variation. Using these inputs, it computes a path metric over minimal routes and directs traffic toward paths with higher relative lifetime. Under PARSEC workloads, Clotho reports average MTTF improvements of 66% for EM and 8% for HCI over deterministic routing. However, Clotho relies on explicit path evaluation and "piggybacking" network lifetime information across the network, which introduces computational and memory overhead, and limits scalability.

A different line of work focuses on combining aging mitigation with performance control. Bhardwaj et al. [11] introduces the Traffic Threshold per Epoch (TTpE) metric, which limits how much traffic a stressed router or link should accept during a given runtime interval. Its routing algorithm chooses paths that are both less stressed and less congested, and can insert recovery cycles once a component reaches its threshold. Experimentally, the paper reports reductions of 13% in network latency, 12.17% in EDPPF, and a 10.4% performance improvement over its comparison scheme. However, this method relies on precomputed lifetime models and static thresholds, which may not adapt well to changing runtime conditions or workloads.

Another important direction is online monitoring-based aging awareness. In [12], Ghaderi et al. propose a Centralized Aging Table (CAT) that maps observed router activity to aging degradation. Their method monitors the number of flits and residence time within routers, then periodically updates

routing to avoid highly aged routers by selecting among shortest paths. Using gem5, it reports average improvements of 39% in maximum-aged router degradation and 52% in aging imbalance compared to XY routing, while introducing negligible overhead in latency and EDP. This idea is extended further in AROMa [6], which targets 3D NoCs. It uses a Distributed Centralized Aging Table (D-CAT), with a separate table per layer, and periodically swaps among k-best shortest paths to avoid highly aged routers and allow BTI recovery. Compared to non-aging-aware routing, it improves maximum age by about 33-34% and age imbalance by 61-72%, with the 3D case showing especially strong gains. However, both CAT and D-CAT rely on explicit aging tables and synchronizing aging information across the network, which can introduce memory overhead and limit scalability as the NoC size increases.

2.4 Gaps in Prior Studies

Prior studies on NoC routing algorithms has made significant progress in both fault-tolerant and aging-aware routing. However, there are still important gaps that this thesis aims to address. First, many aging-aware routing methods rely on explicit aging models or tables that require global state and synchronization, which can limit scalability and adaptability to changing conditions. Second, while RL-based fault-tolerant routing has shown promise, it has not been widely applied to aging-aware routing, which also requires dynamic adaptation to runtime conditions. The main trade-off across the literature is between model fidelity and implementation overhead. To this end, this work proposes to fill these gaps by developing a wear-aware routing algorithm that leverages linear approximate reinforcement learning to adaptively steer traffic away from stressed components without relying on explicit aging tables or models, while also maintaining acceptable network performance.

Chapter 3

Aging-Aware Routing Algorithm System Design

This section describes the system-level modeling and design choices used to enable reliability-aware routing in the NoC. The proposed framework integrates dynamic temperature variation, process variation, and aging models for EM and HCI into the routing decision process. These models are derived from prior work in [2] and are combined with a reinforcement learning-based routing policy inspired by LARE [10].

Although other aging mechanisms such as Bias Temperature Instability (BTI) also affect CMOS devices, they are not considered in this work. As discussed in Clotho, BTI degradation is primarily induced when circuits remain underutilized, which can be mitigated using techniques such as *Use it or Lose it* proposed by Kim er al. [13]. Therefore, this work focuses on EM and HCI, which are directly influenced by traffic patterns and are more suitable for routing-level optimization.

Unlike conventional routing schemes that focus only on performance metrics such as latency or throughput, this design explicitly incorporates device-level reliability factors into routing decisions. Temperature and utilization are treated as dynamic runtime inputs, while process variation is modeled as a static property determined during initialization.

3.1 Dynamic Temperature Variation

Temperature is a key factor influencing both EM and HCI aging and is modeled as a spatially varying runtime parameter. In a CMP, temperature is not uniform across the chip, but depends on the activity of individual cores, with each tile exhibiting its own thermal profile. Following Clotho, the chip is modeled as a set of tiles, each associated with a router, and a per-tile temperature map is maintained. The NoC contributes only a small

fraction of total chip power, so temperature is assumed to be primarily driven by core activity rather than router behavior.

Temperature is treated as a slowly varying parameter and is updated periodically (every 10^7 cycles) to reflect realistic thermal dynamics. The temperature values are generated using a random but spatially correlated distribution within a range of 65°C to 85°C, ensuring that neighboring tiles exhibit similar thermal behavior. This temperature map is used directly in the EM and HCI models, allowing routing decisions to avoid thermally stressed regions and reduce localized wear.

3.2 Process Variation (PV)

Process variation (PV) captures fabrication-time deviations in transistor parameters, leading to permanent differences in device characteristics across the chip. Unlike temperature, which varies at runtime, PV is static and determined once during initialization.

Following [2], PV is modeled through variations in threshold voltage (V_{th}) and effective gate length (L_{eff}) for each router. These variations consist of both random and systematic components. The random component represents uncorrelated device-level fluctuations, while the systematic component is spatially correlated, meaning that nearby routers exhibit similar deviations.

The spatial correlation of the systematic component is modeled using a spherical correlation function:

$$\rho(r) = \begin{cases} 1 - \frac{3r}{2\phi} + \frac{r^3}{2\phi^3}, & r \leq \phi \\ 0, & r > \phi \end{cases} \tag{3.1}$$

where (r) is the distance between two routers and (ϕ) defines the correlation range, expressed as a fraction of a die's length at which there is no more within-die correlation between the PV parameters of any two given transistors.

Since both the random and systematic components are modeled as independent Gaussian variables, the resulting variations in (V_{th}) and (L_{eff}) are also normally distributed. The combined variance is given by the sum of the individual variances of these components.

3.3 Electromigration (EM)

Electromigration (EM) is a wearout mechanism caused by the gradual movement of metal ions in interconnect wires under high current density. In NoCs, EM primarily affects the *Vdd* and *GND* connections of long router-to-router links, as these wires experience repeated charge and discharge cycles due to switching activity.

Following [2], EM-induced degradation is modeled at the links, with lifetime expressed in terms of MTTF:

$$MTTF_{EM} \propto \alpha_{link}^{-2} \cdot \exp\left(\frac{E_{a,EM}}{kT}\right) \tag{3.2}$$

where:

- $\alpha_{link} = 0.5 \times U_{link}$ is the link switching activity, derived from link utilization
- T is the local temperature in Kelvin
- $E_{a,EM}$ is the activation energy for electromigration (approximately 0.9 eV)
- k is Boltzmann's constant

In this work, a proportional MTTF value is computed for each link and used as a relative measure of reliability. These values are updated based on runtime utilization and temperature, allowing the routing algorithm to avoid heavily stressed links. Due to its quadratic dependence on activity, EM is highly sensitive to traffic distribution. Even small changes in routing decisions can significantly improve lifetime.

3.4 Hot-Carrier Injection (HCI)

HCI is a transistor-level aging mechanism in which high-energy carriers become trapped in the gate oxide, leading to a gradual increase in threshold voltage (V_{th}) and degradation of transistor performance. Over time, this results in increased delay along critical paths, eventually causing timing violations in router pipelines.

Following [2], HCI degradation is modeled at the routers, incorporating the effects of PV. The MTTF is given by:

$$MTTF_{HCI,PV} \propto \frac{(I_{sub}(V_{th}, L_{eff}))^{-N}}{\alpha_{router}} \cdot \exp\left(\frac{E_{a,HCI}}{kT}\right) \tag{3.3}$$

where:

- $\alpha_{router} = 0.1 \times U_{router}$ is the router switching activity, reflecting that only a fraction of flits (e.g., header flits) exercise the critical path
- T is the local temperature in Kelvin
- $I_{sub}(V_{th}, L_{eff})$ is the subthreshold leakage current, precomputed per router based on process variation
- N is an empirical exponent (approximately 1.5)
- $E_{a,HCI}$ is the activation energy for HCI (typically 0.2–0.4 eV)
- k is Boltzmann's constant

In this work, a proportional MTTF value is computed for each router using precomputed I_{sub} values and runtime activity and temperature. These values guide routing decisions by discouraging traffic through routers that are inherently weaker or experiencing higher stress. Compared to EM, HCI exhibits weaker sensitivity to load balancing due to its linear dependence on activity. However, its interaction with process variation makes it an important factor in identifying vulnerable routers and improving overall network reliability.

Utilization of Lifetime Models

The EM and HCI models described in the previous sections are used to guide routing decisions and evaluate system reliability. During routing, the MTTF of candidate next-hop routers and links is computed based on current utilization and temperature. These values are normalized relative to the network state and used to define the reward function. The routing policy then prefers paths that traverse components with higher MTTF. Throughout simulation, MTTF values are updated periodically as utilization statistics and temperature maps are refreshed. This allows the system to track the evolution of wear across all routers and links as traffic patterns change over time. At the end of simulation, final MTTF values are collected for all network components.

To compare different routing schemes, the Acceleration Factor (AF) is used as a relative lifetime metric. AF measures the lifetime of the system under a given routing policy relative to a reference baseline. In this work, the reference is deterministic DOR, following [2].

$$AF_{EM} = \frac{MTTF_{EM}\big|_{RL}}{MTTF_{EM}\big|_{DOR}}, \quad AF_{HCI} = \frac{MTTF_{HCI,PV}\big|_{RL}}{MTTF_{HCI,PV}\big|_{DOR}} \tag{3.4}$$

Since both simulations share the same process variation and technology constants, unknown parameters and I_{sub} cancel out in the ratio. As a result, AF depends only on runtime factors such as utilization and temperature.

3.5 Reinforcement Learning-Based Routing Policy

Reinforcement Learning is a learning framework in which an agent interacts with an environment to learn a policy that maximizes long-term reward. At each decision step, the agent observes the current *state*, selects an *action*, and receives a *reward* based on the outcome. Over time, the agent improves its decision-making by learning from these interactions. Figure 3.1 illustrates this agent-environment interaction model.

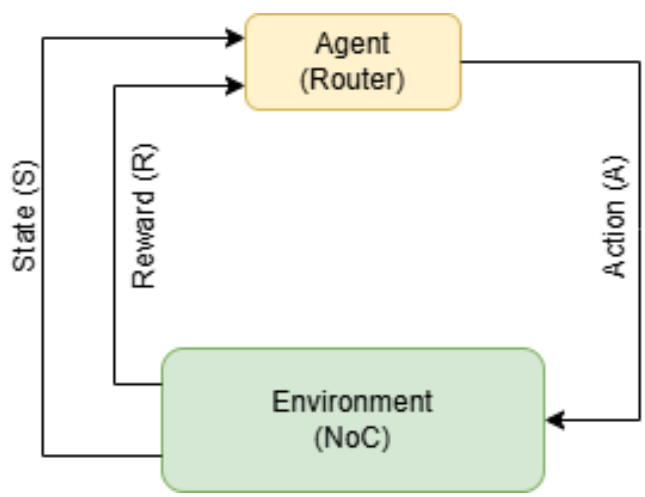


Figure 3.1: Schematic of the reinforcement learning framework for routing policy learning.

In the context of NoCs, each router is modeled as an independent RL agent that selects the next-hop direction for incoming packets. The goal is to learn a routing policy that balances performance and reliability by considering congestion and aging information. This work adopts a linear function approximation-based RL framework inspired by LARE [10], which enables scalable implementation by avoiding large Q-tables.

State Representation

Routing decisions are made at each router when multiple minimal paths are available. The state is represented using a feature vector $\phi(s, a)$ constructed for each candidate action. The feature vector captures both network state and reliability information:

$$\phi(s, a) = [f_0, f_1, f_2, f_3, f_4, f_5, f_6, f_7, f_8] \tag{3.5}$$

where:

- f_0: Bias term (constant 1)
- f_1: Normalized destination router ID
- f_2: Normalized current router ID
- f_3: Normalized hops traversed
- f_4: Normalized remaining distance to destination
- f_5: Normalized EM MTTF of candidate link
- f_6: Normalized HCI MTTF of candidate router
- f_7: Output port congestion level
- f_8: Encoded input port direction

Q-Value Function

To reduce hardware overhead, the Q-value is approximated using a linear function instead of a lookup table:

$$Q(s, a) = \theta \cdot \phi(s, a) \tag{3.6}$$

Each router maintains a parameter vector θ for each output port. This formulation significantly reduces storage requirements compared to tabular Q-learning while preserving learning capability.

Action Selection Policy

An ϵ-greedy policy is used to balance exploration and exploitation:

$$a = \begin{cases} \text{random action}, & \text{with probability } \epsilon \\ \arg\max_a Q(s,a), & \text{otherwise} \end{cases} \tag{3.7}$$

The action space consists of minimal routing directions (horizontal or vertical).

Reward Function

The reward function is designed to capture both reliability and performance objectives:

$$R = 0.7 \cdot \text{MTTF}_{score} - 0.30 \cdot \text{cong}_{norm} + 0.15 \tag{3.8}$$

The reliability component is defined as a weighted combination of normalized EM and HCI metrics:

$$\text{MTTF}_{score} = C_{EM} \cdot EM_{norm} + C_{HCI} \cdot HCI_{norm} \tag{3.9}$$

The normalized MTTF values are computed using peer normalization across candidate actions:

$$EM_{norm} = \frac{MTTF_{EM}}{\max(MTTF_{EM}^{(h)}, MTTF_{EM}^{(v)})} \tag{3.10}$$

$$HCI_{norm} = \frac{MTTF_{HCI}}{\max(MTTF_{HCI}^{(h)}, MTTF_{HCI}^{(v)})} \tag{3.11}$$

To dynamically emphasize the more critical aging mechanism, a weighting function is applied:

$$C(x) = \frac{\exp\left(8 \cdot \frac{x - x_{min}}{x_{max} - x_{min}}\right) - \exp(-8)}{1 - \exp(-8)} \tag{3.12}$$

where $x \in \{MTTF_{EM}, MTTF_{HCI}\}$, and x_{max} and x_{min} are the maximum and minimum values among candidate actions. The weights are then defined as:

$$C_{EM} = C(MTTF_{EM}), \quad C_{HCI} = C(MTTF_{HCI}) \tag{3.13}$$

The congestion term cong_{norm} represents the normalized congestion level of the candidate output port. It is derived from the fraction of active virtual channels (VCs) at the output port and discretized into three levels:

- Low congestion: $\text{cong}_{norm} = 0$
- Medium congestion: $\text{cong}_{norm} = 0.5$
- High congestion: $\text{cong}_{norm} = 1.0$

Update Rule

The Q-value parameters are updated using temporal-difference learning:

$$\delta = R + \gamma \max_{a'} Q(s', a') - Q(s, a) \tag{3.14}$$

$$\theta \leftarrow \theta + \alpha \cdot \delta \cdot \phi(s, a) \tag{3.15}$$

where α is the learning rate and γ is the discount factor.

3.6 Deadlock Avoidance

Adaptive routing algorithms that allow multiple path choices can introduce cyclic dependencies between resources, leading to deadlock. Since the proposed RL-based routing dynamically selects paths based on network conditions, deadlock avoidance must be explicitly enforced. To guarantee deadlock freedom, this work adopts an escape virtual channel (VC) mechanism based on Duato's theory [14]. The key idea is to reserve a subset of resources that follow a strictly deadlock-free routing function.

In this implementation, VC0 is designated as the escape VC and always follows deterministic dimension-order (XY) routing. Since XY routing is provably deadlock-free, packets assigned to VC0 are guaranteed forward progress regardless of network conditions. All other virtual channels are allowed to use the RL-based adaptive routing policy. If a packet cannot safely proceed using adaptive routing, it can fall back to the escape VC, ensuring that no cyclic dependencies persist in the network.

Chapter 4

Evaluation

4.1 Simulation Setup

All simulations are conducted using the gem5 Garnet cycle-accurate NoC simulator [15] [16] in standalone synthetic traffic mode, without a full CPU or ISA model, to isolate network behavior. The network consists of 64 nodes arranged in an 8×8 2D mesh topology. Each node acts as both a traffic source and destination, with packets injected according to configurable synthetic traffic patterns at a specified rate in packets per cycle per node.

The simulated clock frequency is 1 GHz. The router microarchitecture is an input-queued, virtual-channel router with credit-based flow control. Each router has a pipeline latency of 1 cycle and each link has a traversal latency of 1 cycle. The network supports three virtual networks per input port: two control virtual networks (vnets 0 and 1) and one data virtual network (vnet 2). Each virtual network contains 4 virtual channels. Control virtual channels have a buffer depth of 1 flit, while data virtual channels have a buffer depth of 4 flits. Control messages (vnets 0 and 1) consist of a single flit, whereas data messages (vnet 2) consist of 5 flits. The flit width is 128 bits (16 bytes).

The RL-based routing agent is trained over 50 million cycles, using uniform random traffic at an injection rate of 0.2 packets/cycle/node. The learned weight vectors are saved at the end of training run. The learning rate (α), exploration rate (ϵ), and discount factor (γ) are set to 0.01, 0.1, and 0.9, respectively.

For evaluation, the exploration rate is set to $\epsilon = 0$, resulting in a purely greedy policy with no further updates. Both the RL-based routing scheme and the DOR baseline are evaluated across a range of injection rates. Each configuration is simulated for 50 million cycles across multiple synthetic traffic patterns, including uniform random, neighbor, transpose, shuffle, bit reverse, and bit rotation.

The wearout state used in the RL reward is updated periodically during simulation. Link and router utilization are sampled every 1,000 cycles as a windowed average. The spatially correlated temperature map is regenerated every 10^7 cycles. Process variation parameters (ΔV_{th}, ΔL_{eff}) are sampled once at initialization, ensuring identical per-router I_{sub} values across both RL and DOR runs.

4.2 Garnet Synthetic Traffic and Evaluation Metrics

To evaluate the proposed routing approach under a wide range of network conditions, simulations are conducted using seven synthetic traffic patterns: uniform random, neighbor, transpose, tornado, shuffle, bit reverse, and bit rotation. These patterns generate diverse spatial load distributions, resulting in different stress profiles across routers and links and near 100% network utilization.

Uniform random traffic selects destinations uniformly, producing a balanced load across the network. Neighbor traffic restricts communication to adjacent nodes. Transpose traffic maps each node at position (i, j) to (j, i). Tornado traffic sends packet from a node (i, j) in a NxM mesh to a destination node which is offset by roughly half the circumference of the mesh. Shuffle, bit reverse, and bit rotation are structured permutation patterns derived from bitwise transformations of node indices, which create asymmetric and pattern-dependent congestion across the network.

For each traffic pattern, injection rates are swept to capture both low-load and high-load regimes. For uniform random and neighbor traffic, injection rates range from 0.01 to 0.40 packets/cycle/node in increments of 0.03. For the remaining patterns, injection rates range from 0.01 to 0.20 in increments of 0.01. Each configuration is simulated for 50 million cycles.

The evaluation focuses on both performance and reliability metrics. Performance is characterized using average packet latency and throughput, while reliability is assessed using the AF derived from EM and HCI lifetime models. Together, these metrics provide a comprehensive view of the trade-offs between network efficiency and long-term wear.

4.3 Results

Acceleration Factor

To summarize the reliability improvements across traffic patterns, the mean Acceleration Factor (AF) for the weakest 10% of components is presented using bar charts (Figures 4.1 and 4.2). Detailed AF trends as a function of injection rate are provided in the appendix A.

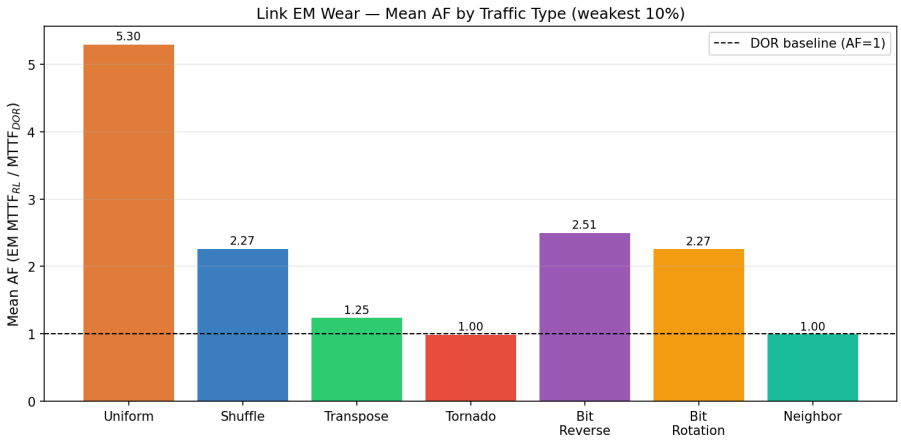


Figure 4.1: Mean Acceleration Factor (AF) for the weakest 10% of links under EM wear across different traffic patterns.

Overall, the proposed RL-based routing achieves significant improvements in lifetime compared to DOR, with consistently higher AF values across most traffic patterns. As expected, the improvement is more pronounced for EM than for HCI due to the quadratic dependence of EM lifetime on switching activity. Redistributing traffic away from heavily utilized links therefore yields substantial gains in EM lifetime, whereas HCI, with its linear dependence and sensitivity to process variation, exhibits more modest improvements.

For EM wear (links), the largest improvement is observed under uniform random traffic, where AF reaches approximately 5.3×. This reflects the ability of the RL policy to effectively balance load in a scenario with high path diversity. Shuffle and bit rotation traffic patterns yield similar improvements of approximately 2.3×, while bit reverse achieves around 2.5×. In contrast, transpose, neighbor, and tornado traffic show significantly lower gains, with AF values of approximately 1.25× for transpose and close to 1.0× for tor-

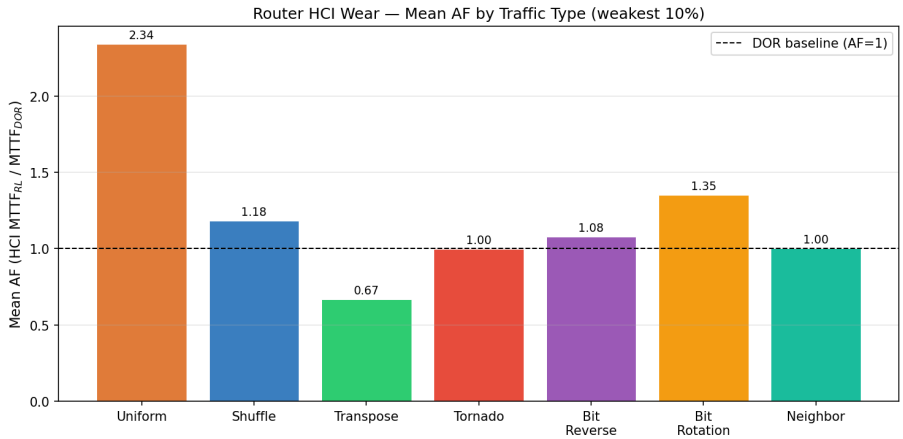


Figure 4.2: Mean Acceleration Factor (AF) for the weakest 10% of routers under HCI wear across different traffic patterns.

nado and neighbor. This is expected, as these traffic patterns constrain packets to a limited set of shortest paths, forcing traffic through central or local links where routing flexibility is reduced.

For HCI wear (routers), a similar trend is observed, though with lower overall gains. Uniform random traffic again provides the highest improvement, with an AF of approximately 2.3×. Shuffle and bit rotation yield moderate improvements of around 1.2×. Bit reverse shows a slight improvement, while tornado and neighbor traffic exhibit little to no gain over DOR. In particular, the RL routing algorithm performs slightly worse under transpose traffic, as the routing options repeatedly stress the same set of routers along fixed shortest paths.

These results highlight two key observations. First, the effectiveness of RL-based routing is strongly dependent on the availability of path diversity. Traffic patterns that allow multiple routing choices benefit the most from adaptive routing. Second, EM benefits more than HCI due to its higher sensitivity to load redistribution, making it the dominant factor in overall lifetime improvement.

Latency Analysis

To evaluate the performance impact of the proposed routing scheme, average packet latency is measured as a function of injection rate across all traffic

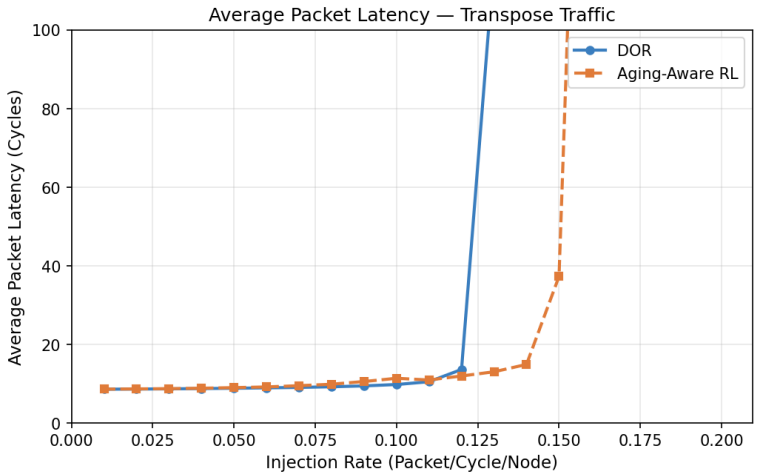
patterns. Representative latency curves are shown in Figures 4.3, 4.4, 4.5, 4.6.

Across all traffic patterns, the RL-based routing achieves latency comparable to or better than the DOR baseline. At low injection rates, both routing schemes exhibit nearly identical latency. This is expected, as the network is lightly loaded and there is limited contention, leaving little opportunity for adaptive routing to improve performance.

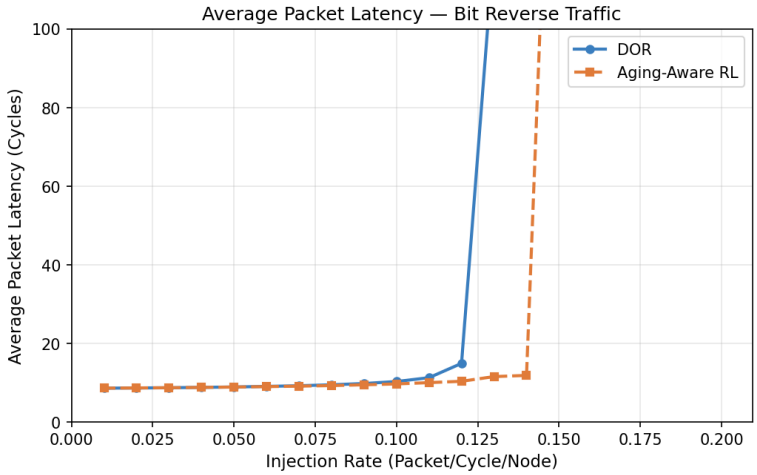
At higher injection rates, the benefits of RL-based routing become more apparent. As traffic increases, DOR begins to saturate due to its deterministic nature, which concentrates traffic along fixed paths and creates congestion hotspots. This results in a sharp increase in packet latency beyond the saturation point. In contrast, the RL-based routing distributes traffic more evenly across available paths, delaying the onset of congestion and maintaining lower latency under higher load conditions.

The most significant latency improvements are observed in structured traffic patterns such as bit reverse, bit rotation, shuffle, transpose, and uniform random. These patterns provide multiple routing options, allowing the RL agent to effectively balance load and avoid congested regions. In these cases, RL not only improves reliability but also enhances network performance.

For neighbor and tornado traffic, both routing schemes exhibit similar latency. This is expected, as these patterns constrain packets to a limited set of shortest paths, leaving little flexibility for adaptive routing to redistribute traffic. As a result, RL does not provide a noticeable performance advantage in these scenarios, but importantly, it does not degrade performance either.



(a) Average Packet Latency under Transpose Traffic.



(b) Average Packet Latency under Bit Reverse Traffic.

Figure 4.3: Average Packet Latency vs. Injection Rate for RL-based Routing and DOR across Transpose, Bit Reverse, and Bit Rotation traffic patterns.

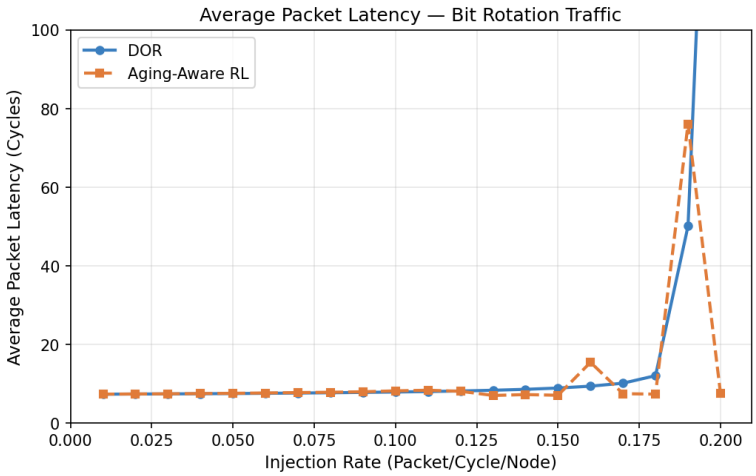
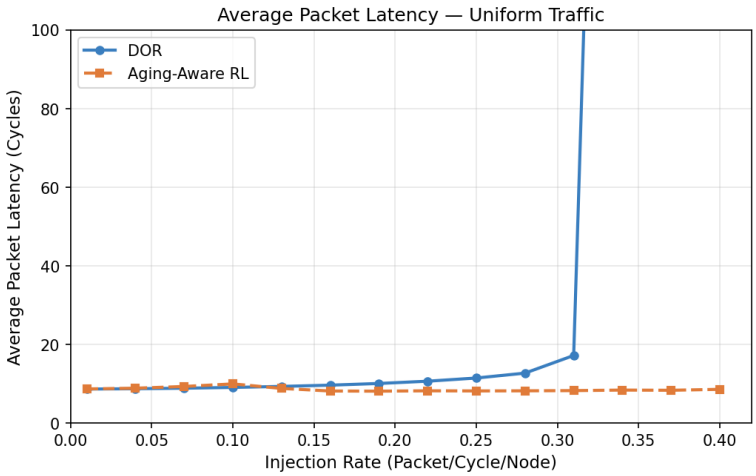
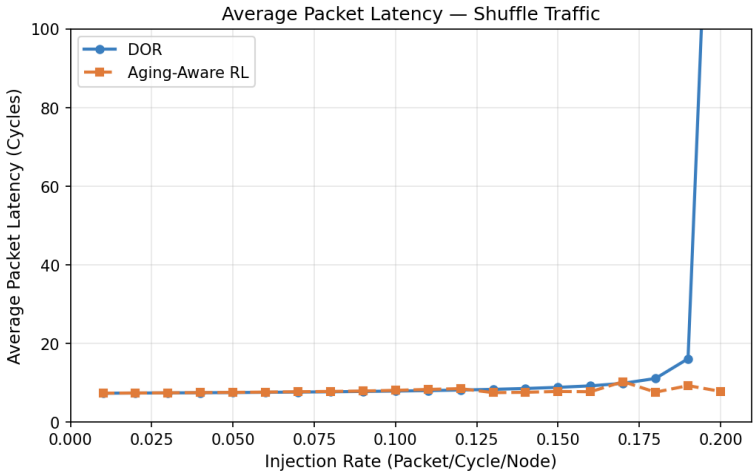


Figure 4.4: Average Packet Latency under Bit Rotation Traffic.

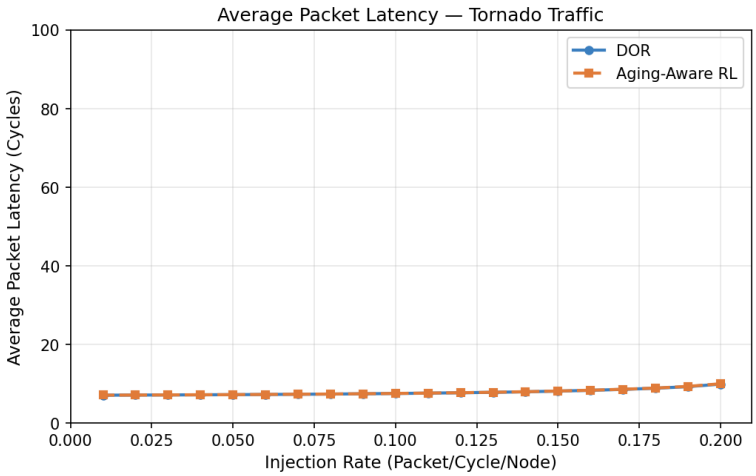


(a) Average Packet Latency under Uniform Random Traffic.

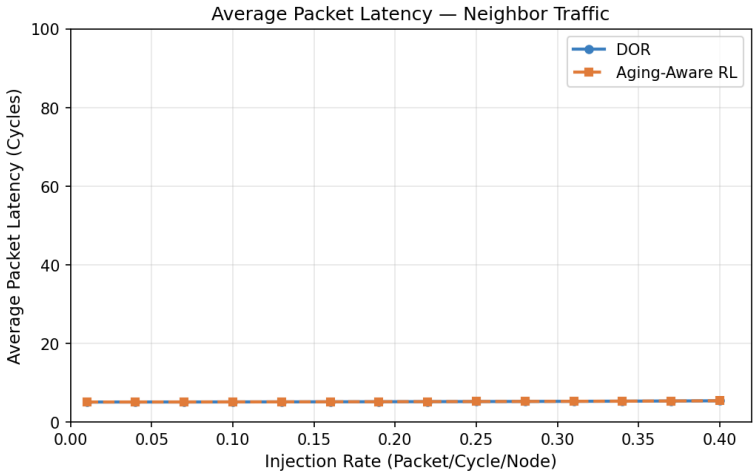


(b) Average Packet Latency under Shuffle Traffic.

Figure 4.5: Average Packet Latency vs. Injection Rate for RL-based Routing and DOR across Uniform Random and Shuffle traffic patterns.



(a) Average Packet Latency under Tornado Traffic.



(b) Average Packet Latency under Neighbor Traffic.

Figure 4.6: Average Packet Latency vs. Injection Rate for RL-based Routing and DOR across Tornado and Neighbor traffic patterns.

Bibliography

[1] W.J. Dally and B. Towles. Route packets, not wires: on-chip interconnection networks. In *Proceedings of the 38th Design Automation Conference (IEEE Cat. No.01CH37232)*, pages 684–689, 2001.

[2] Arseniy Vitkovskiy, Vassos Soteriou, and Paul V. Gratz. Clotho: Proactive wearout deceleration in chip-multiprocessor interconnects. In *2015 33rd IEEE International Conference on Computer Design (ICCD)*, pages 117–124, 2015.

[3] Muhammad Adnan, Muhammad Akmal Chaudary, Muhammad Moazzam Ali, Hafiz Ayaan Aatif, Muhammad Ibrahim Raza, and Hafiz Arslan Ramzan. Fault-tolerant adaptive routing in nocs: Machine learning approaches for resilient on-chip networks. In *2025 International Conference on Artificial Intelligence, Computer, Data Sciences and Applications (ACDSA)*, pages 1–7, 2025.

[4] Jagadheesh Samala, Harshvardhan Takawale, Yash Chokhani, P. Veda Bhanu, and Soumya J. Fault-tolerant routing algorithm for mesh based noc using reinforcement learning. In *2020 24th International Symposium on VLSI Design and Test (VDAT)*, pages 1–6, 2020.

[5] Samala Jagadheesh, P. Veda Bhanu, J. Soumya, and Linga Reddy Cenkeramaddi. Reinforcement learning based fault-tolerant routing algorithm for mesh based noc and its fpga implementation. *IEEE Access*, 10:44724–44737, 2022.

[6] Zana Ghaderi, Ayed Alqahtani, and Nader Bagherzadeh. Aroma: Aging-aware deadlock-free adaptive routing algorithm and online monitoring in 3d nocs. *IEEE Transactions on Parallel and Distributed Systems*, 29(4):772–788, 2018.

[7] Lashmi Kondoth, Rajan Shankaran, Quan Z. Sheng, Endrowednes Kuantama, and Wei Ni. A survey of adaptive routing techniques in

two-dimensional network-on-chip architectures. *IEEE Communications Surveys & Tutorials*, 28:3195–3234, 2026.

[8] Jianfeng An, Hongjun You, Jinhua Sun, and Jiawei Cao. Fault tolerant xy-yx routing algorithm supporting backtracking strategy for noc. In *2021 IEEE Intl Conf on Parallel & Distributed Processing with Applications, Big Data & Cloud Computing, Sustainable Computing & Communications, Social Computing & Networking (ISPA/BDCloud/SocialCom/SustainCom)*, pages 632–635, 2021.

[9] Jagadheesh Samala, Harshvardhan Takawale, Yash Chokhani, P. Veda Bhanu, and Soumya J. Fault-tolerant routing algorithm for mesh based noc using reinforcement learning. In *2020 24th International Symposium on VLSI Design and Test (VDAT)*, pages 1–6, 2020.

[10] Shaocong Wang, Xiaoyun Zhang, Dezun Dong, Cunlu Li, Zicong Wang, and Zongmao Zhang. Lare: A linear approximate reinforcement learning based adaptive routing for network-on-chips. In *2023 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2023.

[11] Kshitij Bhardwaj, Koushik Chakraborty, and Sanghamitra Roy. Towards graceful aging degradation in nocs through an adaptive routing algorithm. In *DAC Design Automation Conference 2012*, pages 382–391, 2012.

[12] Zana Ghaderi, Ayed Alqahtani, and Nader Bagherzadeh. Online monitoring and adaptive routing for aging mitigation in nocs. In *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, pages 67–72, 2017.

[13] Hyungjun Kim, Arseniy Vitkovskiy, Paul V. Gratz, and Vassos Soteriou. Use it or lose it: Wear-out and lifetime in future chip multiprocessors. In *2013 46th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 136–147, 2013.

[14] J. Duato. A new theory of deadlock-free adaptive routing in wormhole networks. *IEEE Transactions on Parallel and Distributed Systems*, 4(12):1320–1331, 1993.

[15] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell,

Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The gem5 simulator. *SIGARCH Comput. Archit. News*, 39(2):1–7, August 2011.

[16] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, Adrià Armejach, Nils Asmussen, Brad Beckmann, Srikant Bharadwaj, Gabe Black, Gedare Bloom, Bobby R. Bruce, Daniel Rodrigues Carvalho, Jeronimo Castrillon, Lizhong Chen, Nicolas Derumigny, Stephan Diestelhorst, Wendy Elsasser, Carlos Escuin, Marjan Fariborz, Amin Farmahini-Farahani, Pouya Fotouhi, Ryan Gambord, Jayneel Gandhi, Dibakar Gope, Thomas Grass, Anthony Gutierrez, Bagus Hanindhito, Andreas Hansson, Swapnil Haria, Austin Harris, Timothy Hayes, Adrian Herrera, Matthew Horsnell, Syed Ali Raza Jafri, Radhika Jagtap, Hanhwi Jang, Reiley Jeyapaul, Timothy M. Jones, Matthias Jung, Subash Kannoth, Hamidreza Khaleghzadeh, Yuetsu Kodama, Tushar Krishna, Tommaso Marinelli, Christian Menard, Andrea Mondelli, Miquel Moreto, Tiago Mück, Omar Naji, Krishnendra Nathella, Hoa Nguyen, Nikos Nikoleris, Lena E. Olson, Marc Orr, Binh Pham, Pablo Prieto, Trivikram Reddy, Alec Roelke, Mahyar Samani, Andreas Sandberg, Javier Setoain, Boris Shingarov, Matthew D. Sinclair, Tuan Ta, Rahul Thakur, Giacomo Travaglini, Michael Upton, Nilay Vaish, Ilias Vougioukas, William Wang, Zhengrong Wang, Norbert Wehn, Christian Weis, David A. Wood, Hongil Yoon, and Éder F. Zulian. The gem5 simulator: Version 20.0+, 2020.

Appendix A

Mean Acceleration Factor

A.1 Mean Acceleration Factor (AF) Trends Across Injection Rates

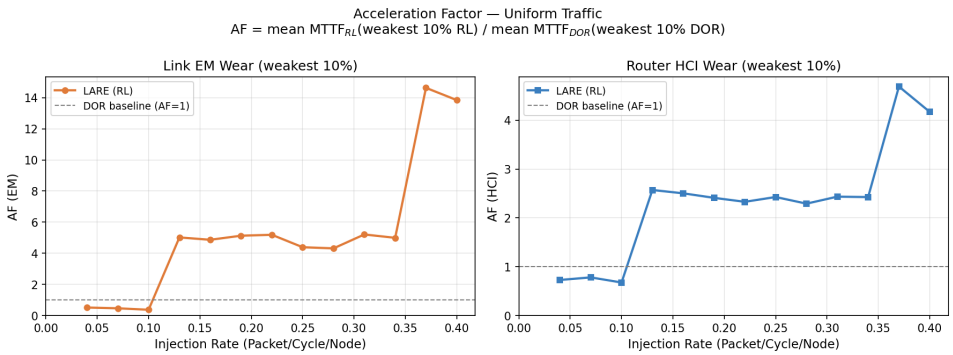


Figure A.1: Mean Acceleration Factor (AF) for uniform random traffic across different injection rates.

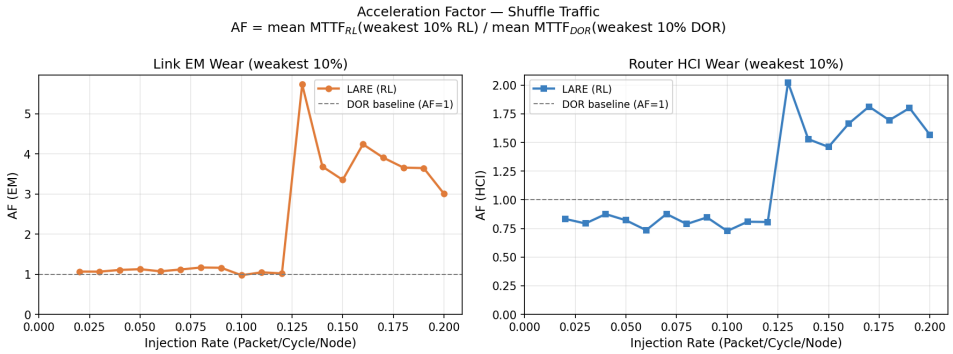


Figure A.2: Mean Acceleration Factor (AF) for shuffle traffic across different injection rates.

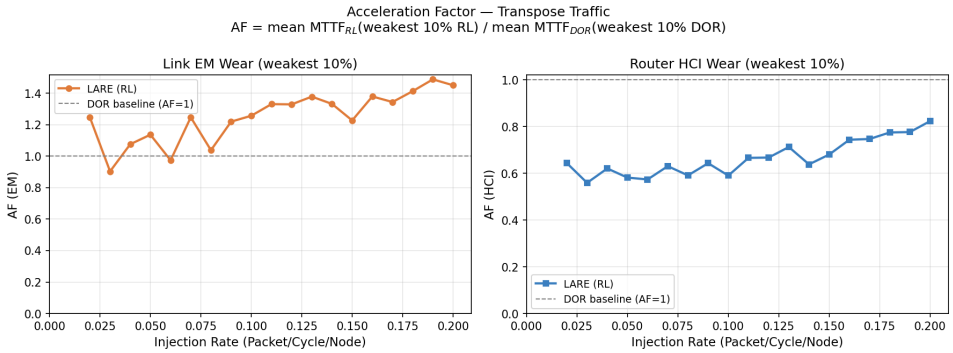


Figure A.3: Mean Acceleration Factor (AF) for transpose traffic across different injection rates.

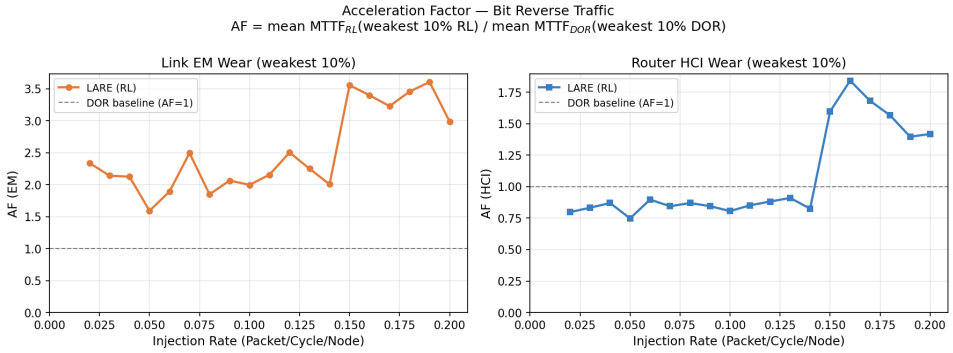


Figure A.4: Mean Acceleration Factor (AF) for bit reverse traffic across different injection rates.

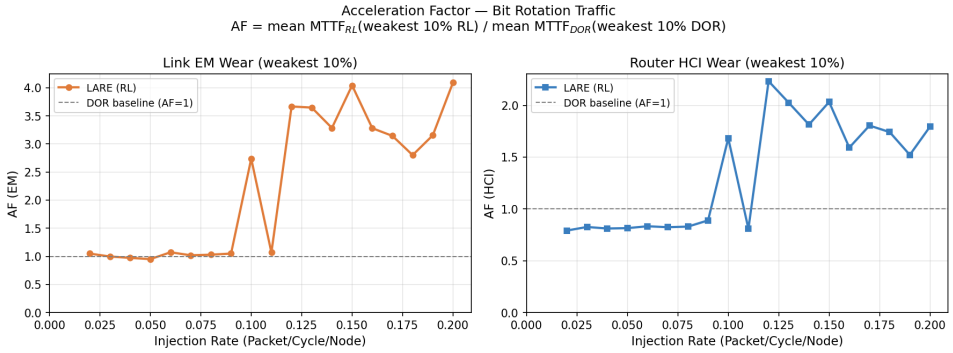


Figure A.5: Mean Acceleration Factor (AF) for bit rotation traffic across different injection rates.

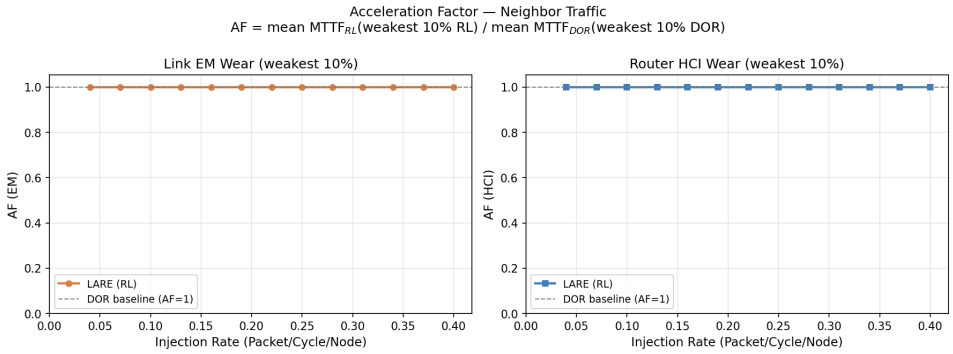


Figure A.6: Mean Acceleration Factor (AF) for neighbor traffic across different injection rates.

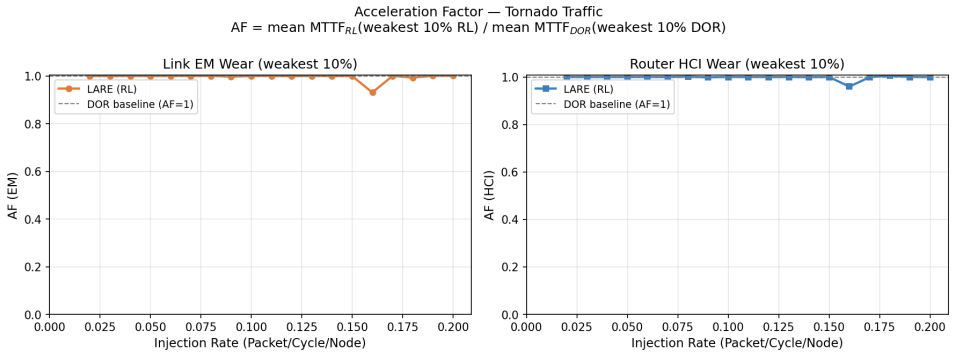


Figure A.7: Mean Acceleration Factor (AF) for tornado traffic across different injection rates.